

Relazione Tecnica: Modello AI per la Zero Trust Architecture

Analisi del Temporal Graph Network (Graphagate)

Capitolo 1

Il modello Graphagate: Temporal Graph Network

1.1 Introduzione

Nel contesto di una Zero Trust Architecture (ZTA), il processo di autorizzazione continua richiede meccanismi dinamici per rilevare deviazioni dal comportamento abituale e segnali di compromissione. Il modello integrato con l'Orchestrator OPA è basato su una *Temporal Graph Network* (TGN), progettata specificamente per il *self-supervised anomaly detection* su stream di traffico continuo.

Ogni accesso (una richiesta IP/device verso una risorsa) rappresenta un arco temporale all'interno di un grafo in evoluzione. Il modello mantiene uno stato ricorrente per ciascuna entità (utente o risorsa) e analizza i vicini temporali recenti per assegnare uno score di anomalia in tempo reale, consentendo di bloccare le richieste ostili ed esfiltranti.

1.2 Architettura del sistema

L'architettura del modello combina la memoria ricorrente tipica dei TGN (Rossi et al., 2020) con un gestore del vicinato ottimizzato (bounded in RAM) e uno *scorer* a due teste che permette di analizzare le anomalie su due fronti: contestuale/policy e strutturale.

1.2.1 Componenti principali

- *TGNMemory*: struttura basata su GRU che aggiorna lo stato storico di ciascun nodo con i messaggi degli eventi in ingresso. La dimensione è impostata a 256 per gestire le impronte comportamentali.
- *Hashed Identity*: identità dei nodi mappate tramite funzioni di hash (`hash(URI) % hash_buckets`) per garantire totale *induttività* e scalabilità in presenza di entità mai viste prima.
- *MessageNeighborLoader*: buffer in RAM che memorizza gli ultimi vicini temporali di ciascun nodo (es. `neighbor_size=30`). Lavora con lookup in tempo $O(1)$ su matrici preallocate ed elimina la necessità di lenti database a grafo.
- *GraphAttentionEmbedding*: rete multi-hop con *TransformerConv* per calcolare l'embedding attendendo sui vicini storici, concatenando l'identità alla storia temporale.
- *Static Node Features*: vettore a 16 dimensioni associato a ciascun nodo, che codifica attributi ZTA essenziali come *device tier*, *clearance* e *trust score*, oltre a metriche estese introdotte nello schema v3 quali il *resource risk* (per innalzare la sensibilità su asset

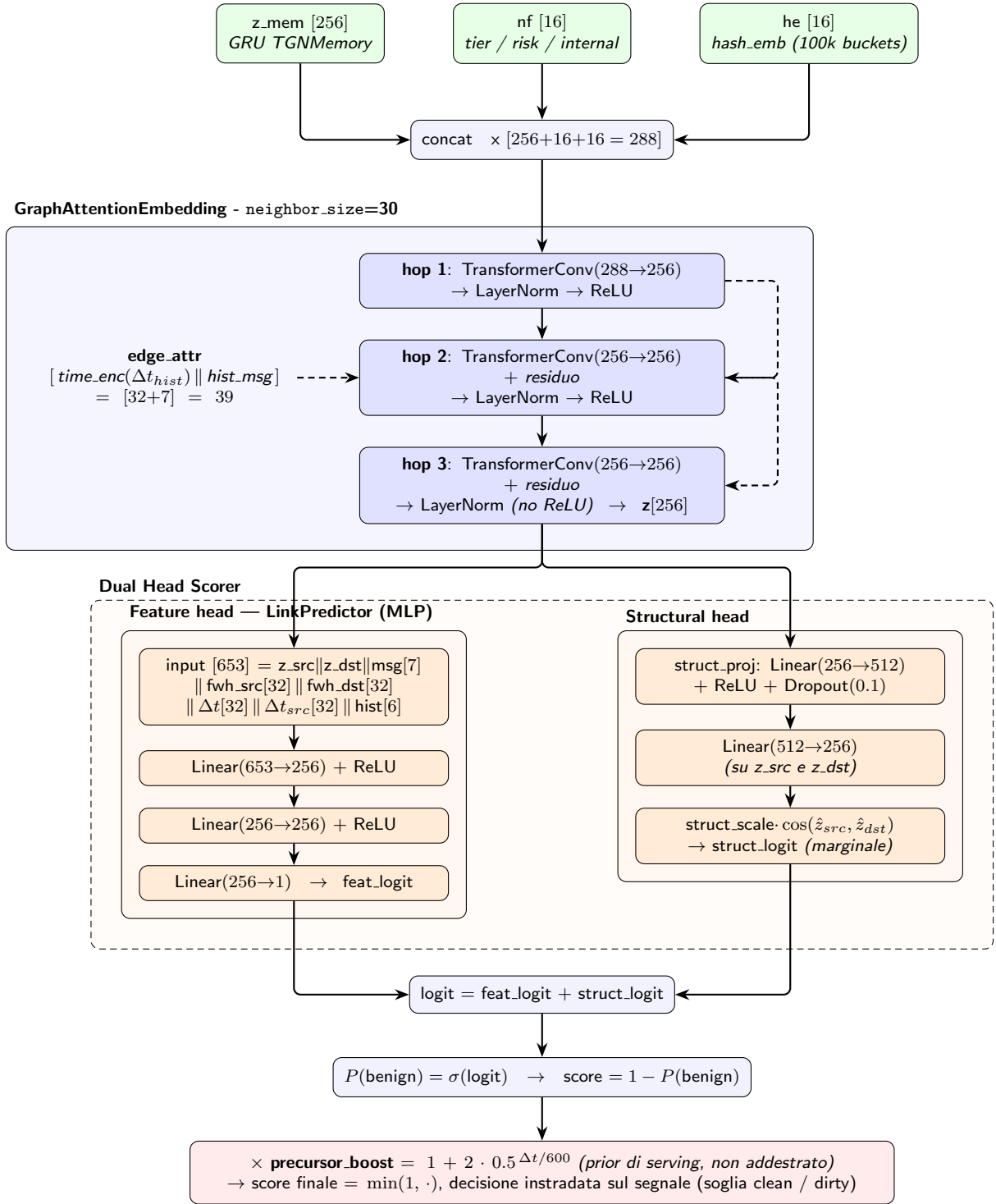


Figura 1.1: Schema dei layer dove i moduli interni sono esplicitati per i due di alto livello (GAE ed il Dual Head Scorer): i tre hop dentro *GraphAttentionEmbedding* e le due teste (*Feature/Structural*) dentro il *Dual Head Scorer*. Tutti i *TransformerConv* usano **heads=4**, **dropout=0.1** e concatenano **edge_attr** (alimenta ogni hop). Lo score grezzo $1 - P(\text{benign})$ è moltiplicato dal prior di kill-chain (con cap a 1) e confrontato con la soglia instradata sul segnale; a serving lo score dell'evento è il massimo sui suoi archi (accesso e binding).

critici) e il *source internal/external flag* (per distinguere reti RFC1918 da IP pubblici; ablabile e *disattivato di default* nella configurazione corrente, poiché il roaming esterno benigno può mascherare il furto di credenziali — la coppia *external*→*device* che esporrebbe l’attaccante).

- *History Features* (esplicite, non circolari): per ogni arco valutato si calcolano tre contatori causali e *benign-gated* (aggiornati solo su traffico approvato, anti-poisoning): $\log(1+\text{pair_count})$, $\log(1+\text{src_count})$, $\log(1+\text{dst_count})$. A questi si aggiunge una seconda tripletta identica per le coppie *aux_src*→*dst* (es. per validare la *device habituality*), portando la dimensione a 6. Misurano quanto è abituale quell’accesso: una coppia mai vista da una sorgente attiva è la *novelty cue* del movimento laterale, che, combinata con la memoria temporale, porta il maggior contributo (ablation multi-seed a 3 seed: +0.163 AUC, il segnale dominante del laterale).
- *Dual Head Scorer*: lo score di un arco è la *somma* di due logit complementari, $\text{logit} = \text{feat_logit} + \text{struct_logit}$.
 - *Feature head* (**LinkPredictor**, MLP a 3 strati): consuma gli embedding dei due endpoint, il messaggio dell’evento, gli attributi statici con hashed-identity, le codifiche di recency (Δt coppia, Δt sorgente) e le *history features*. Cattura anomalie *contestuali* e di *policy* e veicola la novelty cue del laterale.
 - *Structural head*: similarità coseno (scalata da una temperatura apprendibile) delle proiezioni non-lineari degli embedding. *Nota*: l’ablation a 3 seed la mostra *marginale* sul laterale (0.882 con vs 0.875 senza, +0.007 AUC); il segnale laterale è portato in realtà dalle *history features* e dalla memoria temporale, non da questa testa (candidata a semplificazione).
- *Kill-chain Precursor* (prior di serving, non addestrato): il movimento laterale segue tipicamente un alert di recon (Snort / anomalia rilevata) sulla *stessa* entità, ma il gate predict-then-update scarta quel precursore dalla memoria TGN. Si mantiene quindi uno stato per-entità **recent_alert** e si applica un fattore *moltiplicativo* allo score, $1 + \text{max_boost} \cdot 0.5^{\Delta t / \text{half_life}}$ (default $\text{max_boost}=2$, $\text{half_life}=600$ s). Moltiplicativo per costruzione: alza uno score già sospetto sopra soglia senza trasformare in falsi positivi gli score benigni ≈ 0 . Il suo contributo è però *marginale* (ablation multi-seed a 3 seed: +0.013 AUC), comparabile a quello della testa strutturale: il segnale laterale è dominato dalle *history features* e dalla memoria temporale. Non costituisce un input addestrato (il training solo-benigno lo renderebbe una feature morta).

1.3 Fase di addestramento e calibrazione

Il modello viene addestrato con un approccio *self-supervised* unicamente su traffico benigno, senza mai essere esposto a etichette di anomalia (unsupervised anomaly detection). Lo stream temporale sintetico generato viene suddiviso cronologicamente in tre subset: addestramento (70%), validazione (10%) e test (20%).

L’obiettivo di apprendimento per la rete è il *link prediction*: il TGN è istruito per massimizzare lo score assegnato ad eventi realmente accaduti nella baseline benigna e minimizzare contemporaneamente lo score associato ad eventi artificialmente manipolati e generati a runtime (*negative sampling*).

1.3.1 Negative sampling e funzione di loss

Il processo di ottimizzazione (guidato dall’algoritmo AdamW) somma tre termini self-supervised. La componente dominante è una loss di ranking InfoNCE, allineata all’Average Precision, affiancata da due ancoraggi BCE:

- *Structural Negatives* (InfoNCE, $K=5$ random target): per ogni positivo benigno la sorgente è accoppiata a K destinazioni *uniformemente casuali* sull'intero id-range delle destinazioni. L'obiettivo InfoNCE impone che, tra $\{\text{dst-vera}, K \text{ dst-casuali}\}$, la dst-vera ottenga il logit più benigno *dato lo storico della sorgente*. La rete apprende così $P(\text{dst} \mid \text{src}, \text{storia})$: un accesso a bassa verosimiglianza (movimento laterale o violazione di policy) riceve uno score elevato.
- *Positive BCE Anchor*: i logit dei positivi benigni vengono spinti verso “benigno”. L'InfoNCE fissa solo l'ordine relativo; questo ancoraggio mantiene gli score su una scala assoluta, così che la soglia calibrata sull'FPR resti significativa.
- *Contextual Negatives* (Gaussian Feature Noise): mantenendo intatta la destinazione, al messaggio dell'evento si somma rumore gaussiano ($\sigma=0.5$) e si addestra la *Feature Head* a marcarlo come anomalo. Si tratta di un meccanismo distinto rispetto alle anomalie contestuali della valutazione (randomizzazione discreta dei segnali 0/1), così il modello non riceve la corruzione del test.

De-circularizzazione (nessun data leak nei negativi). I negativi strutturali usano *solo* l'id-range delle destinazioni, mai la matrice degli autorizzati né i set abituali del generatore. Una versione precedente impiegava un “hard negative” ristretto agli *autorizzati-ma-non-abituali* (con peso $\times 10$): essendo *la definizione esatta* del movimento laterale di test, produceva un recall circolare e gonfiato. È stato rimosso di proposito (vedi `_sample_structural_negatives`); ogni miglioria deriva da architettura/segnale, mai da negativi che rispecchiano la regola di test. Inoltre il trust (`node_feat[:, 14]`) non viene più mutato dalle etichette in training, evitando di accoppiare la label in una feature di input.

1.3.2 Calibrazione della soglia (cost-sensitive instradata sul segnale)

Il TGN non emette una classificazione netta ma uno score continuo confrontato con una soglia, calibrata via *replay* sul 10% di validazione held-out (lo split di test non è mai toccato in calibrazione). Poiché a serving la classe vera è ignota ma il *segnale di edge* (JA3/Snort/sensori) è osservabile, si calibrano e si instradano due soglie:

- *Soglia signal-dirty* (eventi il cui segnale già scatta): il classico quantile al Target FPR dell'1% sugli score benigni. Questi eventi sono già intercettati dalla cheap rule baseline, quindi si mantiene la soglia conservativa.
- *Soglia signal-clean* (dove il laterale è indistinguibile dal benigno se non per il pattern temporale): soglia *cost-sensitive* che minimizza $\text{cost_ratio} \cdot \text{FN} + \text{FP}$ (default `cost_ratio=20`: un laterale mancato costa molto più di un falso allarme ri-sfidabile dall'orchestrator). Un *cap* esplicito sull'FPR del flusso clean (`clean_fpr_cap=0.05`) impedisce che la spinta sul recall faccia esplodere i falsi positivi — necessario perché anche con lateral AUC ~ 0.90 (singola run) il trade-off recall/FPR sul flusso clean resta ripido (a recall laterale $\sim 79\%$ corrisponde un FPR benigno $\sim 13\%$).

A inferenza la decisione è instradata per-evento: `signal_dirty` $\Rightarrow$ soglia dirty, altrimenti soglia clean (cost-sensitive). Entrambe le soglie sono persistite ed esposte da `serve_api` (anche su `/health`). OPA resta il decisore finale allow/deny.

1.4 Tipologie di anomalie rilevate in produzione

In fase di valutazione o in ambiente live, l'architettura rileva 3 macro-categorie di attacco, dimostrando vantaggi misurabili rispetto a baseline più semplici (rule-based, Isolation Forest, One-Class SVM e GNN non temporale; cfr. Sezione ??):

1. *Policy Anomaly*: l’entità agisce al di fuori del proprio ruolo, clearance o device tier. Intercettata agevolmente dalla Feature head grazie alle feature statiche ZTA.
2. *Contextual Anomaly*: la connessione ha caratteristiche insolite o ha innescato sensori IPS. Rilevata dalla Feature head decostruendo le *edge features*.
3. *Lateral Movement*: l’accesso è formalmente autorizzato a livello di regole OPA, ma *non abituale*. Avviene tipicamente nel “pivoting”. Essendo indistinguibile sintatticamente dal traffico benigno, la classificazione ricade quasi del tutto sull’architettura implementata.

1.5 Valutazione sperimentale e confronto con le baseline

La valutazione è condotta sullo split di test (20% finale dello stream sintetico, `seed=42`) tramite replay per-evento attraverso l’esatto codice di serving (`num_events=200 000`, di cui il 20% finale = 40 000 eventi di test). Per quantificare il contributo *effettivo* della componente temporale, il TGN è confrontato con tre baseline non supervisionate addestrate sullo *stesso* traffico benigno, con identico split cronologico, identici segnali tabellari (contatori di storia + prior precursore), soglia calibrata all’1% di FPR e protocollo di valutazione (codice in `tests/baselines/`); una quarta baseline *supervisionata* (XGBoost) è riportata solo come upper-bound di riferimento:

- *Isolation Forest*: detector non relazionale sulle sole feature statiche per-evento (nessuna struttura di grafo) — rappresenta il “pavimento” privo di informazione strutturale (lateral AUC 0.639).
- *One-Class SVM*: controparte kernel non relazionale dell’Isolation Forest (sole feature per-evento). Pur con AUC aggregata competitiva (0.845), sul *lateral* resta a 0.632, confermando che il segnale temporale non è catturabile dalle sole feature statiche.
- *GNN non temporale*: ablation del TGN su grafo statico aggregato, con lo *stesso* curriculum di negative sampling de-circularizzato (InfoNCE su destinazioni casuali + rumore contestuale) e gli *stessi* contatori tabellari di storia, ma priva di memoria ricorrente e vicinato temporale. Isola la sola componente temporale (lateral AUC 0.468, $\approx$ caso).
- *XGBoost (supervisionato, fuori categoria)*: addestrato con le etichette, raggiunge lateral AUC 0.942. Non è un confronto equo col paradigma non supervisionato del TGN; è incluso solo per delimitare l’upper-bound raggiungibile avendo accesso alle label (assenti per costruzione nel nostro setting).

I risultati del confronto sono riportati nella Tabella ??.

1.5.1 Cost-sensitive routing

Per convertire la forte capacità di ranking (lateral AUC 0.900 su singola run; cfr. il caveat multi-seed in Tab. ??) in recall operativo, è stato implementato un routing cost-sensitive. Il flusso di eventi viene separato a runtime in *signal-dirty* e *signal-clean*. Sul flusso pulito, la soglia viene rilassata per minimizzare i falsi negativi dei movimenti laterali (configurabile tramite un parametro di `cost_ratio`). Sul test sintetico, ciò porta il recall laterale dal $\sim 13.0\%$ (soglia globale all’1% FPR, FPR benigno $\sim 2.3\%$) al $\sim 28.6\%$ (decisione instradata, FPR benigno $\sim 6.0\%$), con recall aggregato $\sim 70.3\%$.

Metrica (test set)	TGN	GNN non temp.	One-Class SVM	Isol. Forest
AUC aggregata	0.947	0.574	0.845	0.775
AP aggregata	0.870	0.569	0.793	0.628
Lateral — AUC	0.900	0.468	0.632	0.639
Lateral — Recall @1%FPR	0.130	0.139	0.047	0.010

Tabella 1.1: Confronto sul test set sintetico (replay per-evento de-circularizzato, `num.events=200000`, `seed=42`, soglia globale all'1% FPR; valori da *singola* run). Il vantaggio del TGN sul *lateral movement* emerge isolando il segnale temporale: la GNN statica, priva di memoria ricorrente ma provvista degli stessi contatori tabellari, resta a livello di caso (AUC 0.468), contro l'AUC 0.900 del TGN. Il recall laterale a soglia globale resta basso (13.0%) e l'apparente 13.9% della GNN è *spurio* (AUC≈caso con soglia permissiva, non un segnale reale); la sua conversione in recall operativo è affidata alla calibrazione cost-sensitive instradata (Sez. 4.3). *Nota metodologica*: il lateral AUC headline è da singola run (training GPU non-deterministico, ± 0.01 – 0.03 per run); la conferma *multi-seed* (3 seed, media $\pm$ dev.std) dà lateral AUC 0.882 ± 0.012 , coerente con l'headline a singola run. XGBoost *supervisionato* (vede le etichette, fuori categoria) raggiunge AUC 0.977 / lateral 0.942: è un upper-bound di riferimento, non una baseline non supervisionata comparabile.

1.6 Deployment e dettagli hardware

Le primitive di model-serving (estrazione memoria ed espansione grafo storico) operano asintoticamente a tempo costante $O(1)$. Il TGN mantiene tutto il suo stato dinamico e ricorrente nativamente in RAM. Ciò comporta due scelte architetturali di deployment: è inefficace una scalabilità orizzontale a monte (poiché più repliche frammenterebbero il database comportamentale delle entità) ed è fondamentale il dimensionamento asincrono.

Il sistema finale eroga un'infrastruttura single-worker fondata su **FastAPI** (gestione I/O asincrono per polling concorrentiale) che elabora gli eventi con una latenza end-to-end (round-trip HTTP client→servizio sulla rete container, percorso completo a 3 archi *source*→*device*→*user*→*dst*) di $P50 \approx 6.0$ ms ($P99 \approx 6.7$ ms); il percorso ablato senza *device-edge* scende a $P50 \approx 4.4$ ms ($P99 \approx 5.0$ ms). Misure su GPU RTX 5070 Ti (VRAM occupata ~ 2.4 GB), tramite test client di streaming live in regime *cold-start* (entità mai viste prima, prefisso `prod_`): un sanity-check d'induttività più che un punto operativo, poiché in cold-start il modello opera ad alto recall ma bassa specificità (lateral recall $\sim 47\%$ con *device-edge* vs $\sim 45\%$ senza, a fronte di una specificità benigna $\sim 76\%$ — FPR gonfiato dall'assenza di storico). Il binding di dispositivo dà un contributo marginale ma coerente; il punto operativo affidabile resta quello calibrato offline (Sez. ??).

Al fine di calcolare simultaneamente la backward-propagation e smaltire la mole della *Graph Attention*, il container isolato si appoggia alle seguenti specifiche hardware/software:

- container engine nativo agganciato a librerie NVIDIA CUDA 13;
- hardware accelerato su GPU con architettura NVIDIA Blackwell (testato su RTX 5070 Ti). La GPU accelera la *Graph Attention* multi-hop in inferenza e la backward-propagation in addestramento; lo stato ricorrente della `TGNMemory` e i ring-buffer del `MessageNeighborLoader` sono mantenuti con accesso $O(1)$ su tensori preallocati.

Nel momento di chiusura del framework (o tramite specifici endpoint API persistenti), la memoria serializzata di PyTorch (`tnn_checkpoint.pt` e `tnn_stats.json`) va copiata su sistema persistente per evitare cold-start traumatici.

1.7 Limiti e sfide implementative

1.7.1 La sfida del lateral movement

Senza l’ausilio delle etichette (unsupervised), il lateral movement non mostra tracce o indicatori palesi: è feature-identico a un accesso benigno non abituale. Le leve che lo rendono *ordinabile* (lateral AUC 0.90 su singola run — 0.882 ± 0.012 in media multi-seed a 3 seed su configurazione ridotta, coerente con la singola run; da ~ 0.47 della GNN statica) sono il vicinato temporale ampio (num.hops=3, neighbor.size=30, memory_dim=256) e soprattutto le *history features* esplicite (+0.163 AUC, il contributo dominante), affiancate dall’hashed-identity (+0.046 AUC); il precursore di kill-chain (+0.013 AUC) e la testa strutturale (+0.007 AUC) risultano invece *marginali* (ablation multi-seed). La conversione del ranking in recall operativo avviene poi con la soglia cost-sensitive instradata (Sezione 4.3): sul test sintetico il recall laterale passa da $\sim 13.0\%$ (soglia globale all’1% FPR, FPR benigno $\sim 2.3\%$) a $\sim 28.6\%$ (decisione instradata, FPR benigno $\sim 6.0\%$) — operating point regolabile via `cost_ratio/clean_fpr_cap`. Resta la classe più ostica (AP ~ 0.44), ma nettamente sopra le baseline (cfr. Sezione ??).

1.7.2 Anti-poisoning gate in live stream

Il design offline del testing replica 1:1 il routing live, prevenendo fenomeni di *data leakage*. Tuttavia, l’ambiente di deployment espone alla minaccia di *data poisoning*. Un aggressore persistente potrebbe infiltrarsi immettendo attacchi lenti per abituare il grafo ai propri comportamenti nocivi, sfalsando lentamente lo scoring e la similarità del coseno. Il problema è mitigato dal gate anti-avvelenamento: sia l’internal state della `TGNMemory` che le code a scorrimento dei vicini temporali vengono aggiornate e *committate* in RAM solo nel caso in cui lo score della rete si posizioni sotto il limite calcolato per le minacce ($< threshold$). I pacchetti classificati come anomalia, inviati a OPA, vengono ignorati dal sistema di Intelligenza Artificiale, preservando incontaminato lo stato vettoriale del modello.